Sorting Algorithms

Radix Sort

Core Idea

Instead of comparing whole numbers, **sort digit by digit** — starting from the least significant digit (rightmost) to the most significant digit (leftmost).

At each digit position, use **Counting Sort** as the subroutine to sort stably. After processing all digit positions, the array is fully sorted.

Why Least Significant Digit First?

If you sorted from the most significant digit first, earlier sorts would get disrupted by later ones. Starting from the **least significant digit and moving left** means each new sort refines the previous one without breaking it — previous order is preserved because Counting Sort is **stable**.

Steps

1. Find the **max value** to know how many digits to process
2. Start from the **ones place** (least significant digit)
3. **Stable sort** all elements by the current digit using Counting Sort
4. Move to the **tens place**, repeat
5. Move to the **hundreds place**, repeat
6. Continue until you've processed all digit positions

Visualizing It

```
[170, 45, 75, 90, 802, 24, 2, 66]
```

Pass 1 — Sort by ones digit:

```
170 → 0
 90 → 0
802 → 2
  2 → 2
 24 → 4
 45 → 5
 75 → 5
 66 → 6
```

Result: `[170, 90, 802, 2, 24, 45, 75, 66]`

Pass 2 — Sort by tens digit:

```
802 → 0
  2 → 0
 24 → 2
 45 → 4
 66 → 6
 170 → 7
 75 → 7
 90 → 9
```

Result: `[802, 2, 24, 45, 66, 170, 75, 90]`

Pass 3 — Sort by hundreds digit:

```
  2 → 0
 24 → 0
 45 → 0
 66 → 0
 75 → 0
 90 → 0
```

```
170 → 1
802 → 8
```

Result: `[2, 24, 45, 66, 75, 90, 170, 802]` ✅

Key Characteristics

- **Stable** — uses Counting Sort at each step which is stable
- **Not in-place** — needs extra arrays at each digit pass
- **Non comparison based** — never compares two numbers directly

---

Time & Space Complexity

Case	Time	Why
Best	O(n × d)	n elements, d digit passes
Average	O(n × d)	Always the same passes
Worst	O(n × d)	Always the same passes

Space: O(n + k) — output array of size n, count array of size k (usually 10 for digits 0–9)

> `d` = number of digits in the maximum value. `k` = base, usually 10.

How it Compares to Counting Sort

	Counting Sort	Radix Sort
Works on	Small range integers	Large integers, strings
Passes	Single pass	One pass per digit
Uses	Directly	Counting Sort as subroutine
Range dependency	Very sensitive to range	Not sensitive, depends on digits

---

When to Use vs When to Avoid

Use When	Avoid When
Large integers with many elements	Floating point numbers
Strings of equal length	Variable length data without padding
Range is too large for Counting Sort	Memory is constrained
Need guaranteed linear time	Small arrays (simpler sorts win)

Why It's Asked in Interviews

Radix Sort is asked to test whether you understand **non comparison based sorting** and how algorithms can be **composed** — it literally uses Counting Sort as a building block. It's also a gateway to deeper questions about **time complexity limits** — breaking the O(n log n) barrier and when that tradeoff is actually worth it in practice.

Code Logic

Why are we separating negative and positive, then re-combining?

Because `(num // exp) % 10` breaks down for negative numbers in Python.

```
-374 // 10  = -38   # Python floors towards negative infinity
-374 % 10   = 6     # gives wrong digit!

# What you'd expect:
374 // 10   = 37
374 % 10    = 4     ✅
```

So `-374` would be sorted by digit `6` instead of `4` — compl
etely wrong bucket.

Why not just fix the digit extraction?

```
You could do `abs(num) // exp % 10`, but then ordering breaks
too:
```
negatives = [-1, -5, -3]

Sorted as positives: 1, 3, 5
But as negatives: -1, -3, -5 ← reverse order!

-1 is the LARGEST negative, -5 is the SMALLEST
so sorted order should be: -5, -3, -1
```

Negatives are a **mirror image** of positives — bigger absolute value means smaller actual value.

---

## So the cleanest approach is:

```
Split
negatives   = [-x for x in nums if x < 0]   # flip to positi
ve → [1, 5, 3]
non_negatives = [x for x in nums if x >= 0]

# Sort both as positive numbers
sorted_pos = radixSort(non_negatives)   # [1, 3, 5]
sorted_neg = radixSort(negatives)   # [1, 3, 5]

# Flip and reverse negatives
sorted_neg = [-x for x in reversed(sorted_neg)] # [-5, -3, -
1]

Combine
return sorted_neg + sorted_pos
```
```

```
negatives=[-5,-3,-1]    non_negatives=[2,4]
     ↓ flip                    ↓
  [5, 3, 1]               [2, 4]
     ↓ radix sort              ↓ radix sort
  [1, 3, 5]               [2, 4]
     ↓ flip+reverse            ↓
  [-5,-3,-1]              [2, 4]
            ↓ combine
      [-5, -3, -1, 2, 4]  ✅
```

In short: separating them lets radix sort work purely on positive integers, which it was designed for, then you stitch results back together.

---

1. Why `count = [0] * 10`?

Any number in base 10 can only have digits **0 through 9** at any position — nothing else is possible.

```
374  →  digits are 3, 7, 4   all between 0-9
9021 →  digits are 9, 0, 2, 1 all between 0-9
```

So we only ever need exactly **10 buckets**, regardless of how large the numbers are.

---

2. Why `% 10`?

`% 10` isolates the **last digit** of any number.

```
374 % 10 = 4   ✅ ones digit
37  % 10 = 7   ✅ ones digit
3   % 10 = 3   ✅ ones digit
```

---

3. How does `exp` navigate digit by digit?

`// exp` first **shifts** the number right, then `% 10` **isolates** the digit:

```
num = 374

exp=1   → 374 // 1   = 374 → 374 % 10 = 4   (ones)
exp=10  → 374 // 10  = 37  → 37  % 10 = 7   (tens)
exp=100 → 374 // 100 = 3   → 3   % 10 = 3   (hundreds)
exp=1000→ 374 // 1000 = 0  → loop stops     (no more digits)
```

Visually:

```
   3  7  4
   ↑  ↑  ↑
exp=100 10  1
```

Putting it all together — full trace on `[374, 121, 423]`

Pass 1 — sort by ones digit (exp=1):

```
374 → digit = 4
121 → digit = 1
423 → digit = 3

count = [0,1,0,1,1,0,0,0,0,0]
           ↑     ↑ ↑
           1     3 4

after prefix sum:
count = [0,1,1,2,3,3,3,3,3,3]

output = [121, 423, 374]  ✅ sorted by ones
```

Pass 2 — sort by tens digit (exp=10):

```
121 → digit = 2
423 → digit = 2
374 → digit = 7
```

```
output = [121, 423, 374]  ✅ sorted by tens (121,423 both hav
e 2, stability preserves order)
```

Pass 3 — sort by hundreds digit (exp=100):

```
121 → digit = 1
423 → digit = 4
374 → digit = 3

output = [121, 374, 423]  ✅ fully sorted!
```

Key insight: each pass builds on the previous one — stability ensures earlier digit orderings are never lost as you move left.